

Batch Code: ANP_D1838

Student ID: AF05202193

Student Name: S. Vaishnavi

LAB-03

Question:

1. What is constructor & explain about its types.
 2. Create default constructor with example program - [comments, output screenshots]
 3. Create parameterized constructor with example program - [comments, output screenshots]
-

1. Constructor:

A Constructor is a special method in Java used to initialize the object. When an object is created, the constructor is automatically executed. In simple words, a constructor is used to assign values to class variables during object creation.

Characteristics:

- Constructor name and class name must be same.
- Constructor does not have any return type (not even void).
- It is automatically called when an object is created using new keyword.
- Mainly used for initializing variables.

Syntax:

```
ClassName ()  
{  
    // initialization code  
}
```

Types of Constructors:

1. Default Constructor

A constructor without parameters is called a Default Constructor. It does not receive any values from the user. It is used to assign fixed or default values to class variables.

Features:

- Takes no arguments.
- Used to assign default values.
- Automatically called when an object is created.

Syntax:

```
ClassName ()  
{  
    // default values  
}
```

2. Parameterized Constructor

A constructor that accepts one or more parameters is called a Parameterized Constructor. It allows values to be passed during object creation. It is used when different objects need different values.

Features:

- Accepts values as arguments.
- Used to initialize variables with user-defined values.
- Helps to create objects with different data.

Syntax:

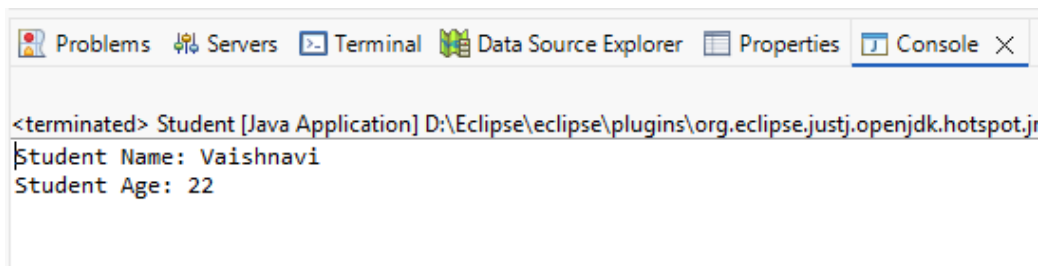
```
ClassName (datatype variable)  
{  
    // initialization using parameters  
}
```

2. Default Constructor:

Program:

```
public class Student {  
    int age;  
    String name;  
    Student ()  
    {  
        name = "Vaishnavi";  
        age = 22;  
    }  
    void getInfo ()  
    {  
        System.out.println("Student Name: " + name);  
        System.out.println("Student Age: " + age);  
    }  
    public static void main (String [] args) {  
        Student s1 = new Student ();  
        s1. getInfo ();  
    }  
}
```

Output:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Servers, Terminal, Data Source Explorer, Properties, and Console. The console output displays the following text:
<terminated> Student [Java Application] D:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jr
Student Name: Vaishnavi
Student Age: 22

Software Used: Eclipse IDE

File Name: Student.java

Comments:

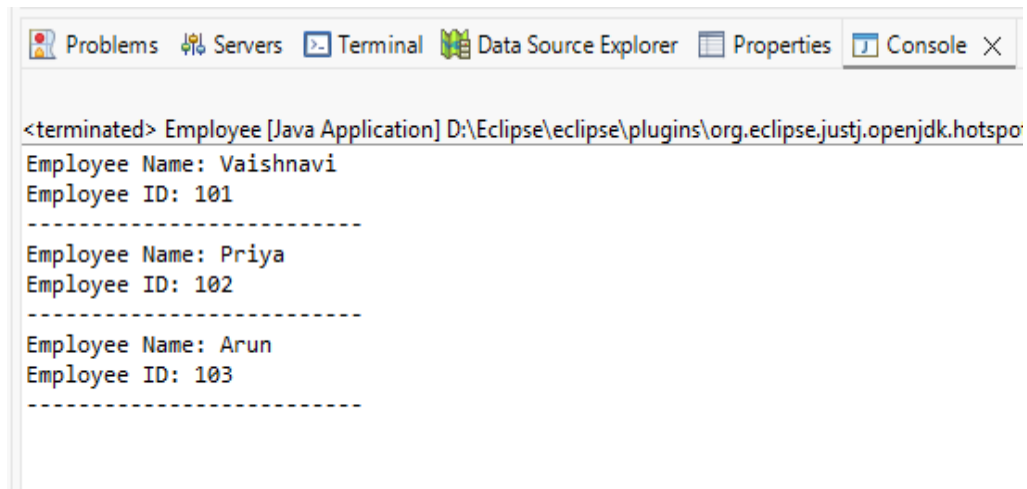
public class Student	public → class can be accessed from anywhere. class → used to create a class. Student → class name.
int age;	int → integer datatype. age → class variable to store age.
String name;	String → datatype used for text values. name → class variable to store student name.
Student ()	Default Constructor. Constructor name and class name are same. No parameters are used.
name = "Vaishnavi";	Assigns the default value "Vaishnavi" to the name variable.
age = 22;	Assigns the default value 22 to the age variable.
void getInfo ()	void → no return value. getInfo () → method used to display information.
System.out.println();	Prints the line in the console output.
public static void main (String [] args)	Main method — starting point of Java program execution.
Student s1 = new Student ();	Creates object s1. new Student () automatically calls the default constructor.
s1. getInfo ();	Calls getInfo () method using object s1

3. Parameterized Constructor:

Program:

```
public class Employee {
    String name;
    int empId;
    Employee (String empName, int id)
    {
        name = empName;
        empId = id;
    }
    void getInfo ()
    {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + empId);
        System.out.println("-----");
    }
    public static void main (String [] args) {
        Employee Vaishnavi = new Employee("Vaishnavi",101);
        Employee Priya = new Employee("Priya",102);
        Employee Arun = new Employee("Arun",103);
        Vaishnavi.getInfo();
        Priya.getInfo();
        Arun.getInfo();
    }
}
```

Output:



```
<terminated> Employee [Java Application] D:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotsp
Employee Name: Vaishnavi
Employee ID: 101
-----
Employee Name: Priya
Employee ID: 102
-----
Employee Name: Arun
Employee ID: 103
-----
```

Software Used: Eclipse IDE

File Name: Employee.java

Comments:

public class Employee	public → class can be accessed from anywhere. class → used to create a class. Employee → class name.
String name;	String → datatype used for storing text values. name → class variable to store employee name.
int empId;	int → integer datatype. empId → class variable to store employee ID.
Employee (String empName, int id)	Parameterized Constructor. Constructor name and class name are same. String empName and int id are parameters used to receive values.
name = empName;	Assigns parameter value empName to class variable name.
empId = id;	Assigns parameter value id to class variable empId.

<code>void getInfo ()</code>	void → no return value. getInfo () → method used to display employee details
<code>System.out.println();</code>	Prints the line in the console output.
<code>public static void main (String [] args)</code>	Main method → starting point of Java program execution.
<code>Employee Vaishnavi = new Employee("Vaishnavi",101);</code>	Creates object Vaishnavi. "Vaishnavi" and 101 are passed as arguments to the parameterized constructor.
<code>Employee Priya = new Employee("Priya",102);</code>	Creates object Priya and passes values to constructor.
<code>Employee Arun = new Employee("Arun",103);</code>	Creates object Arun and passes values to constructor.
<code>Vaishnavi.getInfo();</code>	Calls getInfo () method using Vaishnavi object.
<code>Priya.getInfo();</code>	Calls getInfo () method using Priya object.
<code>Arun.getInfo();</code>	Calls getInfo () method using Arun object.